

Name: Chinta Lakshmi Nivas Jaswanth

Spring Data JPA with Spring Boot, Hibernate

1) Spring Data JPA - Quick Example

application.properties

```
# Spring Framework and application log
logging.level.org.springframework=info
logging.level.com.cognizant=debug

# Hibernate logs
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace

# Log pattern
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p %-
25.25logger{25} %25M %4L %m%n

# Database configuration
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root

# Hibernate configuration
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

Country Table

```
CREATE TABLE country (  
    code VARCHAR(2) PRIMARY KEY,  
    name VARCHAR(50)  
);  
  
INSERT INTO country VALUES ('IN', 'India');  
  
INSERT INTO country VALUES ('US', 'United States of America');
```

Country.java

```
package com.cognizant.ormlearn.model;  
  
import jakarta.persistence.Column;  
import jakarta.persistence.Entity;  
import jakarta.persistence.Id;  
import jakarta.persistence.Table;  
  
@Entity  
@Table(name = "country")  
public class Country {  
    @Id  
    @Column(name = "code")  
    private String code;  
    @Column(name = "name")  
    private String name;  
    public String getCode() {  
        return code;  
    }  
    public void setCode(String code) {  
        this.code = code;  
    }  
}
```

```

    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}

```

CountryRepository.java

```

package com.cognizant.ormlearn.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

}

```

CountryService.java

```

package com.cognizant.ormlearn.service;

```

```

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service

public class CountryService {

    @Autowired

    private CountryRepository countryRepository;

    @Transactional

    public List<Country> getAllCountries() {

        return countryRepository.findAll();

    }

}

```

OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication

public class OrmLearnApplication {

```

```

private static final Logger LOGGER =
    LoggerFactory.getLogger(OrmLearnApplication.class);
private static CountryService countryService;
public static void main(String[] args) {
    ApplicationContext context =
        SpringApplication.run(OrmLearnApplication.class, args);
    countryService = context.getBean(CountryService.class);
    LOGGER.info("Inside main");
    testGetAllCountries();
}
private static void testGetAllCountries() {
    LOGGER.info("Start");
    List<Country> countries = countryService.getAllCountries();
    LOGGER.debug("countries={}", countries);
    LOGGER.info("End");
}
}

```

2) Difference between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	Framework / ORM Tool	Framework
Developed By	Java (Jakarta/Oracle)	Hibernate Team	Spring
Implementation	No implementation	Implements JPA	Uses JPA implementations

Feature	JPA	Hibernate	Spring Data JPA
Purpose	Defines persistence standards	Provides ORM functionality	Reduces boilerplate code
SQL Generation	Not directly	Yes	Through Hibernate/JPA
Transaction Management	Not provided	Manual or JPA based	Automatic with @Transactional
Boilerplate Code	More	Moderate	Very Less
Repository Support	No	No	Yes (JpaRepository)

Without Hibernate/JPA

```
Connection con = DriverManager.getConnection(...);
PreparedStatement ps =
    con.prepareStatement(
        "insert into employee values (?, ?)");
ps.setInt(1, 1);
ps.setString(2, "John");
ps.executeUpdate();
```

Using Hibernate

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
session.save(employee);
tx.commit();
session.close();
```

Using SpringDataJPA

Repository:

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {
}
```

Service:

```
@Autowired
```

```
private EmployeeRepository employeeRepository;  
  
@Transactional  
public void addEmployee(Employee employee) {  
    employeeRepository.save(employee);  
}
```